

C++ · Python Reliability Simulation Lab

Level 1-5 실전 포트폴리오 가이드

버전 0.5.0 · 2026-08-19 · CLion + PyCharm + GitHub + LinkedIn

이 문서의 Level은 JetBrains 공식 등급이 아니라, 프로젝트를 실행하고 설명하는 단계부터 시스템을 소유하고 가르치는 단계까지의 자체 역량 기준이다.

목차

1. 사용 방법과 공통 주의사항
2. Level 1 — 실행하고 설명하기
3. Level 2 — 안전하게 확장하기
4. Level 3 — 시스템으로 설계하기
5. Level 4 — 프로덕션 엔지니어링
6. Level 5 — 시스템을 소유하고 가르치기
7. LinkedIn 연결 절차
8. 최종 증거 체크리스트

1. 사용 방법과 공통 주의사항

저장소: <https://github.com/darkwingduck-code/linkedin-simulation-lab>

공개 릴리스: <https://github.com/darkwingduck-code/linkedin-simulation-lab/releases/tag/v0.5.0>

모든 단계는 학습 → 실행 → 검증 → 증거 저장 → 설명 순서로 진행한다. 완료 표시는 코드, 테스트, CI, 결과 파일 또는 문서 링크가 있을 때만 한다.

공통 실행:

```
cd C:\Users\sadva\linkedin-simulation-lab .\venv\Scripts\python.exe -m pip install -e ".[dev]"
.\scripts\run.ps1
```

주의사항:

- 회사 코드, 고객 데이터, API key, session cookie를 저장소에 넣지 않는다.
- artifacts 결과가 교육용 parameter에서 생성됐음을 항상 밝힌다.
- speedup은 장비와 부하에 따라 달라지므로 보장값으로 쓰지 않는다.
- LinkedIn Connect apps와 CLion/PyCharm IDE 통합은 서로 다른 기능이다.
- 실패한 명령은 성공한 것처럼 기록하지 말고 exit code와 원인을 남긴다.

2. Level 1 — 실행하고 설명하기

목표

기존 C++/Python 파이프라인을 두 IDE에서 도움 없이 실행하고, 데이터 흐름과 핵심 함수를 자기 말로 설명한다.

CLion 실습

1. 저장소 루트를 CMake 프로젝트로 연다.
2. Debug와 Release profile에서 `reliability_simulator`, `reliability_tests` 를 빌드한다.
3. `simulate` 에 breakpoint를 걸고 `clock`, `downtime`, `failures` 변화를 관찰한다.
4. 동일 seed로 두 번 실행해 결과가 같은지 확인한다.

```
--output artifacts/level1-debug.csv --runs 1000 --seed 42
```

PyCharm 실습

1. `.venv\Scripts\python.exe` 를 interpreter로 선택한다.
2. `python` 디렉토리를 Sources Root로 지정한다.
3. `python/tests` 를 unittest configuration으로 실행한다.
4. `summarize` 에 breakpoint를 걸고 CSV row와 percentile 계산을 관찰한다.

이해해야 할 내용

C++ simulator는 고장까지의 시간과 복구 시간을 지수분포에서 표본화한다. 각 run의 uptime, downtime, failures, availability를 CSV/JSON으로 저장한다. Python은 파일 계약을 검증하고 평균, 중앙값, 표준편차, 하위 5%, downtime 95분위수를 계산한다.

통과 기준

- Debug/Release build 성공
- CTest와 unittest 성공
- 데이터 흐름과 핵심 함수 설명 문서
- 두 IDE의 debugger를 직접 사용한 기록

흔한 문제

`reliability_lab` 을 찾지 못하면 interpreter, Sources Root, working directory를 순서대로 확인한다. CLion toolchain 오류는 CMake profile과 bundled MinGW/Ninja 경로를 확인한다.

3. Level 2 — 안전하게 확장하기

목표

기존 동작을 깨뜨리지 않고 입력 계약과 분석 지표를 확장한다.

구현 범위

C++ CLI는 `--runs`, `--seed`, `--hours`, `--failure-rate`, `--repair-rate` named option을 사용한다. 모든 수치 입력은 유한성, 양수 범위, 정수 범위를 검증한다. Python은 median, standard deviation, p05 availability, p95 downtime을 산출한다.

```
.\build\reliability_simulator.exe `  --output artifacts\level2.csv `  --runs 5000 --seed 7 --hours 720 `  --failure-rate 0.0015 --repair-rate 0.08
```

검증 관점

- runs 0, rate 0, 음수, NaN/Infinity 거부
- 빈 CSV, 필수 column 누락, 잘못된 숫자 거부
- C++ warning-clean build
- strict mypy 통과
- 두 언어 합계 10개 이상의 집중 테스트

설계 판단

CLI option은 재현 가능한 실험 명세다. 오류 메시지는 사용자가 수정할 수 있는 입력 오류와 내부 실패를 구분한다. CSV는 단순하고 감사하기 쉽지만, in-process 성능이나 강한 schema evolution이 필요하면 pybind11/API/message queue를 다시 검토한다.

통과 기준

입력 검증, 확장 통계, data contract, warning/type gate가 모두 Git과 CI에 남아야 한다.

4. Level 3 — 시스템으로 설계하기

목표

단일 스크립트를 명시적 domain type, versioned contract, scenario comparison, HTML report가 있는 애플리케이션으로 발전시킨다.

구조

- C++: configuration, simulation result, serialization, CLI 책임 분리
- 교환 계약: CSV와 `schema_version: 1.0` JSON
- Python: analytics, scenario comparison, reporting, benchmark
- E2E: executable 실행부터 HTML report 생성까지 자동 검증

```
.\scripts\run-scenarios.ps1
```

생성 결과:

- `artifacts/baseline.csv`, `stressed.csv`
- `artifacts/comparison.json`
- `artifacts/comparison.html`
- benchmark JSON

분석 질문

- seed와 표본 수가 재현성과 sampling uncertainty에 미치는 영향은 무엇인가?
- 언어 경계와 domain 경계는 어떻게 다른가?
- file contract가 적합하지 않게 되는 규모와 latency는 어디인가?
- 입력 오류, infrastructure failure, programmer error를 어떻게 구분하는가?

통과 기준

ADR, benchmark baseline, versioned JSON, chart가 포함된 HTML, automated E2E test, 2분 pipeline demo가 필요하다.

5. Level 4 — 프로덕션 엔지니어링

목표

다른 개발자와 자동화 시스템이 신뢰할 수 있는 build/test/release 구조를 만든다.

품질 게이트

- Windows/Linux GitHub Actions
- CTest, unittest, Ruff, strict mypy
- clang-tidy static analysis
- Linux AddressSanitizer/UndefinedBehaviorSanitizer
- randomized invariant tests
- 50,000-run performance regression gate
- version consistency check

```
python scripts/check_version.py .\scripts\package.ps1
```

배포

v0.4.0부터 Windows executable, Python wheel, source distribution을 GitHub Release에 제공한다. release는 tag, changelog, 재현 절차, CI 성공을 함께 갖춰야 한다.

보안과 장애 대응

CSV/JSON을 신뢰하지 않는 입력으로 취급하고 path, resource exhaustion, dependency risk를 검토한다. Release build에서 `assert`가 제거되는 문제와 native exit code 전달을 incident drill로 기록했다.

통과 기준

운영체제 2개 CI, sanitizer, tagged release, incident drill, 사용자 효과 중심 release note가 모두 성공해야 한다.

6. Level 5 — 시스템을 소유하고 가르치기

문제 정의

1년 동안 repairable service 하나의 availability를 추정하고, failure prevention과 repair acceleration 중 어느 투자를 먼저 조사할지 지원한다. 실제 안전 인증이나 SLA 결정이 아니라 engineering workflow를 검증하는 교육용 capstone이다.

Parameter provenance

항목	기준값	변화 범위	성격
horizon	8,760 h	고정	1년 정의
failure rate	0.0015/h	0.0010–0.0020	교육용 calibrated assumption
repair rate	0.08/h	0.05–0.12	평균 repair time 비교용 가정
runs	100,000	고정	정확도/실행시간 절충
seed	20260818	고정	재현용
threads	4	1과 비교	bounded worker 수

결정론적 병렬화

각 Monte Carlo run에 독립 seed sequence를 부여한다. worker scheduling과 난수 소비 순서를 분리하고 결과 vector slot을 run id 순으로 고정한다. thread 수가 달라도 CSV가 byte-for-byte 같아야 한다. thread 수는 1–256으로 제한하며 생성 중 예외가 발생하면 기존 worker를 join한다.

```
.\scripts\run-capstone.ps1
```

최신 측정 증거

- 100,000 runs serial: 0.6768초
- four workers: 0.2828초
- 최신 재현 speedup: 2.39x
- 연속 측정 범위: 2.39x–3.50x
- serial/parallel exact match: true
- baseline mean availability: 0.981640

- slow repair: 0.970935
- fast repair: 0.987665

Sensitivity와 uncertainty

Python capstone은 failure-low/baseline/failure-high/repair-slow/repair-fast 다섯 scenario를 실행한다. 평균 availability, p05 availability, 평균의 95% confidence interval을 versioned JSON과 HTML/SVG로 남긴다.

Engineering recommendation

교육용 범위에서는 repair가 느려질 때 손실이 크게 나타났다. 따라서 실제 조직은 MTTR 분포와 복구 병목을 먼저 측정하고 repair automation의 비용 대비 효과를 failure prevention과 함께 비교해야 한다. simulator 하나만으로 투자를 확정하지 않는다.

신뢰 금지 범위

- common-cause failure, redundancy, load dependency가 중요한 시스템
- non-stationary/seasonal rate
- 고장과 복구가 독립 지수분포가 아닌 시스템
- 인명·환경 안전과 규제 인증
- 실제 data calibration 없는 SLA 비용 계산
- 평균 availability만으로 tail risk를 설명하는 경우

100배 규모에서의 재설계

CSV I/O와 result memory가 먼저 병목이 될 가능성이 높다. streaming aggregation, chunked output, process-level scenario parallelism을 검토한다.

통과 기준

- self-review 3회
- public v0.5.0 release와 재현 가이드
- test/static analysis/benchmark 증거
- 기술 글과 10분 demo script
- Level 1과 Level 5 비교 retrospective
- LinkedIn Projects 링크는 계정 소유자가 직접 확인

7. LinkedIn 연결 절차

제품 경계

CLion/PyCharm은 LinkedIn 연결 앱이 아니다. ChatGPT/Codex Connect apps는 외부 서비스 connector 목록이고 JetBrains IDE 통합은 IDE의 AI Chat에서 Codex를 선택하는 별도 기능이다. Connect apps에 IDE 이름이 없어도 정상이다.

기본 연결 경로

1. LinkedIn `Me > View Profile`.
2. `Add profile section > Recommended > Projects`.
3. 이름: `C++/Python Reliability Simulation Lab`.
4. 설명에는 Git으로 검증되는 사실만 입력한다.
5. URL field가 없으면 `Add media > Add a link`.
6. 저장소 또는 v0.5.0 Release URL을 입력한다.
7. Save 후 시크릿 창에서 링크를 클릭해 확인한다.

저장소 URL:

```
https://github.com/darkwingduck-code/linkedin-simulation-lab
```

Release URL:

```
https://github.com/darkwingduck-code/linkedin-simulation-lab/releases/tag/v0.5.0
```

Featured가 보이는 계정은 외부 링크를 추가할 수 있다. 보이지 않으면 Projects를 사용한다. Contact info에는 website 보조 링크를 추가할 수 있다.

LinkedIn 주의사항

- 게시글 작성은 필수가 아니다.
- 교육용 parameter를 실제 산업 데이터라고 쓰지 않는다.
- 2.39x를 모든 장비의 보장값처럼 쓰지 않는다.
- 비밀번호, cookie, token을 자동화 script에 넣지 않는다.
- Projects/Featured 링크를 실제 클릭해 보기 전 완료 처리하지 않는다.

공식 참고:

- LinkedIn profile sections: <https://www.linkedin.com/help/linkedin/answer/a540837/add-sections-to-your-profile>
- LinkedIn Featured samples: <https://www.linkedin.com/help/linkedin/answer/a550399/feature-samples-of-your-work-on-your-linkedin-profile>
- OpenAI Codex IDE: <https://learn.chatgpt.com/docs/codex/ide>

8. 최종 증거 체크리스트

코드와 실행

- ☐ CLion Release build와 CTest 직접 실행
- ☐ PyCharm unittest 직접 실행
- ☐ `scripts/run-capstone.ps1` 성공
- ☐ capstone JSON/HTML 확인

GitHub

- ☒ public repository
- ☒ Windows/Linux CI
- ☒ sanitizer/static analysis/package jobs
- ☒ public v0.5.0 release
- ☒ executable/wheel/sdist/JSON/HTML assets

설명 능력

- ☐ 10분 demo를 본인 말로 진행
- ☐ parameter provenance와 limitation 설명
- ☐ deterministic parallelism 설명
- ☐ 100배 규모 병목 설명

LinkedIn

- ☐ Projects에 GitHub link 추가
- ☐ 프로필에서 link 클릭 검증
- ☐ 공개 설명과 저장소 수치 대조

최종 원칙

Level 5는 “모든 것을 안다”는 선언이 아니다. 문제와 가정을 명시하고, 구현을 검증하고, 결과의 불확실성과 한계를 설명하며, 다른 사람이 같은 결론에 도달할 수 있도록 증거를 남기는 능력이다.